

Chapitre 4

Dictionnaire, index et techniques de hachage

4.1 Introduction

Index et dictionnaire sont deux structures de données pour représenter un ensemble de mots et rechercher si un mot appartient ou non à la structure. La différence fondamentale entre ces deux structures c'est les données conservées dans l'une ou l'autre de ces structures, la taille de la structure et la complexité des opérations possibles.

4.2 Dictionnaire

Un dictionnaire est un ensemble de mots qui supporte uniquement les trois opérations suivantes :

1. rechercher un mot
2. insérer un mot
3. supprimer un mot

L'opération de recherche est considérée comme la plus importante. C'est l'unique opération si le dictionnaire est statique, s'il est dynamique, le nombre de consultations (recherche de mots) est généralement très supérieur au nombres d'ajouts et de suppressions de mots, d'où son importance.

Remarque 4.2.1 *De nombreuses applications informatiques font appel à des structures de données du type "dictionnaire". Par exemple en compilation, la gestion de la table des symboles dans le programme consiste à manipuler*

cette table comme un dictionnaire où un symbole est un identificateur dans le programme.

L'implémentation des dictionnaires à l'aide du *hachage* est très efficace. Ces opérations ne nécessitent en moyenne qu'un temps en $O(1)$.

Plus formellement :

Définition 4.2.2 *A tout ensemble de mots fini $E = \{m_1, m_2, \dots, m_x\}$, on associe une structure $\text{Dictionnaire}(E)$ qui, étant donné un mot m permet de vérifier si oui ou non m appartient au dictionnaire. On peut décider de retourner la position $i, i = \{1, 2, \dots, x\}$ de m dans le dictionnaire s'il existe, et 0 sinon.*

4.3 Index

Un index est une structure de données permettant de représenter les occurrences (positions) d'un ensemble de mots d'un texte ou d'un ensemble de textes.

Rechercher un mot dans le texte consiste à s'intéresser à retrouver, soit sa première occurrence, soit toutes ses occurrences, soit ses occurrences à partir d'une position donnée du texte, soit dans un intervalle de positions donné, etc.

L'index est dynamique si on peut ajouter de nouveaux mots (ou textes) à l'index actuel. Les opérations de base dans un dictionnaire sont généralisées dans la structure d'index. Elle se basent aussi sur les techniques du hachage pour une implémentation plus efficace.

4.3.1 Définition

Définition 4.3.1 *Soit $E = \{T_1, T_2, \dots, T_x\}$ un ensemble de x textes. On définit l'index de E par l'ensemble suivant :*

$\text{Index}(E) = \{(m, p, i) \mid m \text{ est un mot ou un facteur dans au moins un } T_i, (i = 1, \dots, x), \text{ et } p \text{ une position de } m \text{ dans un } T_i \in E\}$

Donc si $E = \{T_1, T_2, \dots, T_x\}$ est un ensemble de x textes, l'index de ces x textes est un ensemble de mots, tel que chaque mot a une occurrence dans au moins un $T_i (i = 1, \dots, x)$, et p est la position de cette occurrence.

4.3.2 Opérations

- **La recherche** : "Trouver toutes les positions d'un mot dans un texte"
La principale opération qu'on veut pouvoir appliquer sur un index est l'opération suivante : Étant donné un mot m , trouver l'ensemble de toutes les positions de m dans E , autrement dit,
"trouver l'ensemble des éléments (p, i) tels que $\{(m, p, i) \in \text{Index}(E)\}$ ".
- **La mise à jour** : "Ajouter ou supprimer un texte à l'ensemble des textes"
Dans un index dynamique on veut de plus pouvoir ajouter ou supprimer un texte T à l'ensemble E sans reconstruire l'index depuis le début (incrémentalement). Si l'ensemble des mots ne change pas, l'index est statique, donc les opérations de mise à jour ne sont plus nécessaires.

Les sections qui suivent introduisent les notions de base du hachage.

4.4 Le hachage

4.4.1 Définitions

Définition 4.4.1 *Table de hachage*

Une table de hachage est une structure de données qui permet d'implémenter un dictionnaire. Soit T une table de données (un dictionnaire) et x un élément de T (ou mot du dictionnaire). Une table de hachage permet d'associer une clé d'accès $f(x)$ à chaque élément x de T .

La recherche d'une clé dans une table de hachage est au pire en $\Theta(n)$, n étant le nombre d'entrées de la table. En pratique on arrive à des algorithmes de recherche qui sont en moyenne en $O(1)$ pour un élément qui existe dans la table.

La table de hachage généralise la notion de tableau classique. L'accès direct à un élément i du tableau se fait en $O(1)$. Cette généralisation est rendue possible grâce à la notion de *clé*.

Définition 4.4.2 *Fonction de hachage*

La clé, position de l'élément dans la table T , se calcule à l'aide d'une fonction f , dite fonction de hachage. Cette fonction prend en entrée un élément x de T et fournit en sortie un entier (dans un intervalle donné, par exemple $[0, m]$). La fonction f est construite de manière à ce que les entiers qu'elle fournit soient uniformément distribués dans l'intervalle. Ces entiers sont les indices du tableau T .

Les tables de hachage sont très utiles lorsque le nombre de données effectivement stockées est très petit par rapport au nombre de données possibles. La table de hachage utilise un tableau de taille proportionnelle au nombre de données effectivement stockées. Au lieu d'utiliser la donnée directement comme indice du tableau, cet indice est calculé à partir de la donnée pour retrouver un élément rapidement, sans parcourir le tableau. Cette recherche est en moyenne en $O(1)$.

Soit T un tableau de n entiers tel que $1 \leq T[i] \leq n$. Un algorithme en $O(1)$ consiste à placer les éléments tels que $T[i] = i$.

Exemple 4.4.3 Soit T le tableau suivant :

7	5	4	6	1	8	2	3
---	---	---	---	---	---	---	---

La recherche dans ce tableau est en $O(n)$. Mais si on constate que tous les $T[i] \leq 8$, on peut obtenir une recherche en $O(1)$ avec le tableau suivant :

1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---

Mais si n est grand et le nombre de clés présentes dans T est m avec $m \ll n$, on ne peut plus adopter cette solution, on utilise les tables de hachage. C'est le cas d'un dictionnaire de la langue. L'alphabet est $\Sigma = \{a, b, c, d, \dots, y, z\}$, $|\Sigma| = 26$, le nombre de mots possibles obtenus par combinaisons et permutations des caractères est très grand par rapport au nombre réel des mots d'un dictionnaire. L'arrangement de r lettres parmi 26, donne A_{26}^r mots possibles, soit $\frac{26!}{(26-r)!}$. Si r tend vers 26 (le mot le plus long), ce nombre tend vers $26!$ mots possibles ¹.

Définition 4.4.4 Collision

Si n est la taille de T toute clé qui dépasse n sera calculée modulo n . La fonction f n'est donc pas injective : deux éléments différents x_1 et x_2 de T peuvent avoir la même clé $f(x_1) = f(x_2)$. On dit qu'il y a collision.

Remarque 4.4.5 Une bonne fonction de hachage doit minimiser le nombre de collisions.

Exemple 4.4.6 Soit $Z = \{11, 59, 32, 44, 31, 26, 19\}$ un ensemble d'entiers positifs et T un tableau de taille 10. On range les éléments de Z dans T , en prenant par exemple la fonction de hachage $f(x) = z_i$ modulo 10.

1. $10! = 3\,628\,800$. Rappelons que dans le dictionnaire Robert 2010, il y a 60.000 mots.

z_i	11	59	32	44	31	26	19
$h(z_i)$	1	9	2	4	1	6	9

Il y a collision entre 11 et 31 et aussi entre 59 et 19.

Un algorithme complet de hachage consiste en une fonction de hachage et une méthode de résolution des collisions.

Remarque 4.4.7 Les éléments sont à la fois insérés et recherchés dans la table en utilisant la fonction de hachage.

Il y a deux grandes classes distinctes de méthodes de résolution de collisions.

4.4.2 Hachage par chaînage

Dans cette méthode, les éléments en collision sont chaînés dans une liste. Une fois la clé trouvée, la recherche est séquentielle dans la liste. Elle est linéaire en la taille de cette liste. L'avantage de cette méthode est la simplicité de l'ajout et de la suppression. L'insertion est en $O(1)$ dans le pire des cas (on insère en tête de liste). La suppression est en $O(1)$ si les listes sont doublement chaînées. Si les listes sont simplement chaînées, il faut parcourir pour garder l'adresse du précédent, donc au pire c'est comme la recherche. Au lieu d'une liste, on peut utiliser un arbre binaire équilibré ou un tableau dynamique. Il existe plusieurs autres méthodes de résolution des collisions.

Définition 4.4.8 La complexité en moyenne

Soit A un algorithme et d une donnée, on note $\text{cout}_A(d)$ le nombre d'opérations élémentaires pour traiter la donnée d . La complexité en moyenne de l'algorithme A sur des données de taille n est définie par l'expression suivante :

$$\sum_{d \in D_n} p(d) * \text{cout}_A(d)$$

$p(d)$ est la probabilité pour que la clé d soit tirée.

Définition 4.4.9 Le facteur de remplissage

Soit T une table de hachage de m cases, contenant n éléments. On définit le facteur de remplissage α par le rapport n/m qui représente le nombre moyen d'éléments stockés dans une chaîne.

Pour l'analyse du hachage, on suppose que α reste fixe quand n et m tendent vers l'infini. Le cas pire du hachage par chaînage arrive quand les n clés sont chaînées dans la même liste (les n clés ont la même valeur de fonction de hachage). La recherche est en $\Theta(n)$, (il faut parcourir toute la liste des n clés) plus le temps de calculer les valeurs de la fonction.

La complexité en moyenne du hachage dépend de la manière dont la fonction répartit l'ensemble des clés entre les m valeurs possibles. On suppose que chaque élément a la même probabilité d'être haché dans l'une des cases, indépendamment où les autres éléments ont été hachés (une fonction de hachage uniforme simple).

Remarque 4.4.10 Une bonne fonction de hachage vérifie l'hypothèse de hachage uniforme simple. Si chaque clé est tirée indépendamment de l'ensemble de l'univers des clés selon une loi de probabilité P ($P(k)$ est la probabilité d'avoir la clé k par la fonction), l'hypothèse de hachage simple uniforme signifie que :

$$\sum_{k:f(k)=j} P(k) = \frac{1}{m} \quad \text{pour} \quad j = 0, 1, \dots, m-1.$$

Toute la difficulté est dans la détermination de P souvent inconnue.

Théorème 4.4.11 Dans ce type de hachage, une recherche qui échoue prend en moyenne un temps en $\Theta(1 + \alpha)$

En effet, α est la longueur moyenne des listes. Une recherche qui échoue parcourt une des m listes qui est de longueur α , plus le temps de calcul de la fonction de hachage.

Théorème 4.4.12 Dans ce type de hachage, une recherche qui réussit prend en moyenne un temps en $\Theta(1 + \alpha)$

4.4.3 Adressage ouvert

Dans cette seconde méthode dite " adressage ouvert " (open-addressing) tous les éléments sont conservés dans la table de hachage elle-même. Chaque entrée de la table contient soit une clé, soit NUL. Si k est une clé, on vérifie si $f(k)$ est dans la table ou non, si on ne le trouve pas, on calcule un nouvel indice basé sur la clé, c'est-à-dire *re-hache* dans la table jusqu'à trouver la clé ou NUL si l'élément n'appartient pas à la table.

On peut aussi chaîner dans cette même table les éléments ayant même valeur de fonction de hachage (en collision), mais cette solution est moins

intéressante puisque la recherche implique une partie de recherche séquentielle en suivant les pointeurs.

Dans ce type de hachage, la table peut se remplir entièrement et on ne peut plus insérer une nouvelle clé. Le facteur de remplissage ne doit jamais être supérieur à 1.

Théorème 4.4.13 *Soit une table de hachage (uniforme simple) en adressage ouvert avec un facteur de remplissage $\alpha = n/m < 1$. Le nombre d'éléments examinés pour une recherche qui échoue vaut au plus $1/(1 - \alpha)$.*

Théorème 4.4.14 *Soit une table de hachage (uniforme simple) en adressage ouvert avec un facteur de remplissage $\alpha = n/m < 1$. Le nombre d'éléments examinés pour une recherche qui réussit vaut au plus $\frac{1}{\alpha} * \ln(\frac{1}{1-\alpha})$.*

4.5 Types de hachage

Il y a plusieurs types de hachage dont l'efficacité dépend des contraintes de leur application.

1. Hachage statique

Dans le hachage statique les données sont stockées dans des tables de taille fixe. Un espace mémoire statique est alloué pour contenir la table de hachage. Si l'espace alloué est trop grand on risque une perte d'espace inutile, s'il est insuffisant il faut restructurer les tables.

2. Hachage dynamique

Il permet d'étendre le hachage statique pour s'adapter dynamiquement à des tailles de tables (ou fichiers) de données croissantes ou décroissantes sans pénalité de coût.

3. Hachage parfait

C'est un hachage dans lequel il n'y a pas de collision ; sa fonction est injective.

Pour atteindre la solution idéale d'un nombre nul de collisions, on se base sur l'idée de rendre le hachage aléatoire.

4. Table de hachage distribué

C'est une structure de données qui permet d'identifier et de retrouver une information dans un système réparti, comme les réseaux P2P.

2. DHT : Distributed hash table

Chaque nœud du réseau gère une partie de la table de hachage, il est responsable d'un certain intervalle de clés.

L'utilisation d'un hachage réparti implique qu'il n'y a pas de connaissance globale centralisée de l'ensemble des données. Les ressources sont uniformément réparties entre les nœuds.

L'algorithme de base d'un tel système consiste à retrouver le nœud responsable d'une clé donnée. Les clés et les nœuds reçoivent un identificateur de m bits : l'identificateur nœud provient du hachage de son adresse IP, l'identificateur clé provient du hachage de la clé. La fonction de hachage doit avoir de bonnes propriétés (minimisation des collisions).